On Modern Text-to-SQL Semantic Parsing Methodologies for Natural Language Interface to Databases: A Comparative Study

Moses Visperas[1], Aunhel John Adoptante[1], Christalline Joie Borjal[1], Ma. Teresita Abia[1], Jasper Kyle Catapang[2], Elmer Peramo[1*]
[1]*Computer Software Division, Advanced Science and Technology Institute*
Department of Science and Technology, Diliman, Quezon City, Philippines
[2]*Department of English Language and Linguistics*
University of Birmingham, United Kingdom
*elmer@asti.dost.gov.ph

***Abstract*—NLIDB research has gained popularity recently, mainly as a means of enhancing outcomes and performance. This study makes an effort to give readers background information on how the subject has evolved recently using different text-to-SQL procedures and approaches, as well as an appraisal of the advantages and disadvantages of each methodology. In contrast with past studies, this paper describes the search and selection processes and provide an overview of the complete process for each approach under review before making comparisons. The authors also evaluated the performance of each methodology against a widely recognized benchmark dataset. Along with model performance, each model was compared and assessed based on its overall structure and associated processes, such as using pre-trained language models and intermediate representations. The results of this study show that the field of text-to-SQL semantic parsing has advanced significantly in recent years, as seen by the improved performance of the models under consideration. It was clear that most recent developments concentrated on the encoder side, even if each technique follows an encoder-decoder design. The imbalance opens up much room for decoder advancement in subsequent studies. Using pre-trained language models was also noteworthy for improving the models' performances; the authors will consider this for future efforts. The selection of intermediate representations, on the other hand, is wholly arbitrary.**

***Index Terms*—text-to-SQL, natural language query, natural language processing, semantic parsing, NLIDB**

I. Introduction

In this day and age of rapid technological advancement, a large amount of information is being stored in an increasing number of databases. Relational databases have been the most popular database type since their introduction in the 1970s, and they are critical in all businesses for managing their information on a daily basis [1]. Structured Query Language (SQL) is used daily in many industries, including software development, finance, medicine, and education, to create, delete, update, and read data. SQL is a database query language that provides excellent flexibility and powerful functions for accessing and operating database systems [2]. Non-technical users, on the other hand, face greater difficulties in learning SQL. Writing complex and correct SQL statements for various databases and tables can be difficult, even for technical users.

Interacting with databases and knowledge bases using mainly natural language text is becoming a popular method for querying information. In the face of unclear demands and large amounts of data, it is no longer required to offer a singular and accurate response to a well-phrased and unambiguous question but rather to serve as an "information guide" or smart assistant. The ability to query databases effectively using natural language gives the vast majority of users who are not proficient in query languages access to the power of large datasets. The impact of these databases will be significantly changed if they can be accessed using natural language text queries [3].

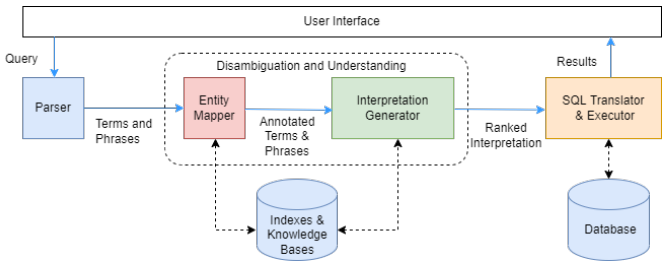

Fig. 1. General architecture of a NLIDB System

The Natural Language Interface to Database (NLIDB) technique uses natural language for the information seeker to submit queries and obtain results from databases, saving the user from learning the database language/schemas [4]. NLIDB takes several steps to transmit the unprocessed text, usually in English, to the database query. This query is then sent to the database in order to retrieve the information as quickly as possible. NLIDB allows users to interact with the database without knowing any additional formal or technical languages [5]. As a result, NLIDBs are designed to output more accurate information and optimize search results. Fig. 1 illustrates the general architecture of a NLIDB system [6].

In recent years, NLIDB research has gained prominence, primarily to improve results and performance. Fig. 2 shows a

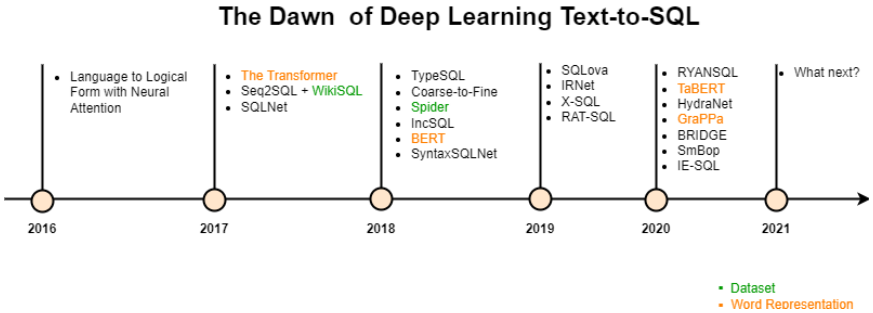

Fig. 2. Timeline of NL2SQL Systems using Deep Learning

simple representation of the progress of the field throughout the years [6]. An increasing number of studies have emerged since the introduction of large datasets, such as WikiSQL [7] and Spider [8]. Numerous articles that use natural language to generate SQL queries have been thoroughly examined and discussed in different papers, such as in the studies of Bukhari et al. [5], Gniki et al. [6], Abbas et al. [9], Iacob et al. [10], and Baig et al. [11]. In fact, Qin et al. [12] recently released a survey study on text-to-SQL parsing that emphasized several principles and techniques, as well as potential future research possibilities.

The authors of this work reviewed a number of papers published in recent years that used various text-to-SQL methodologies and approaches. This study attempts to provide readers with background information on the field's recent development as well as an analysis of each approach's benefits and drawbacks. Unlike prior research, most notably that of Qin et al. [12], this paper presents an overview of the end-to-end process of each methodology studied before drawing comparisons. A benchmark dataset that is widely used in the field was utilized to compare the effectiveness of each methodology. In addition to model performance, each model were compared and evaluated based on its general structure and associated processes, such as the usage of pre-trained language models and intermediate representations.

II. Benchmark

A. Dataset

Datasets are the building blocks for training, assessing, and evaluating machine learning models and have been crucial in the development of the discipline [13]. Additionally, the goals, principles, and research agendas of the machine learning community have been significantly influenced by benchmark datasets. In this section, we examine the different benchmark datasets that are currently available and their different limitations.

WikiSQL: In 2017, Zhong et al. introduced WikiSQL [7] to the world. At the time, WikiSQL was regarded as the largest hand-annotated semantic parsing dataset in the field, considering that it is significantly bigger, in terms of the volume of examples or tables, than existing logical-form datasets. It is composed of 87 726 pairs of natural language questions and their corresponding SQL queries. This huge dataset is then further divided into three: training, development, and test, containing 61 297, 9 145, and 17 284 examples, respectively. Despite the potential WikiSQL holds, it also has some drawbacks. For one, because queries in the dataset are spread over a massive number of tables, it poses the challenge of model generalization not just for brand new queries but for novel table schema as well. WikiSQL only contains simple queries and singular tables, which does not replicate real-world settings.

Spider: In the works of Yu et al., [8], they introduced *Spider*, a fairly large, elaborate, cross-domain text-to-SQL dataset. Their main goal was to launch a challenging, state-of-the-art text-to-SQL and cross-domain semantic parsing task, in which compelex databases and SQL queries comprises both the training and test sets. Compared to the totality of previously released text-to-SQL datasets combined, *Spider* has twice as many nested queries and up to 10 times more `ORDER BY (LIMIT)` and `GROUP BY (HAVING)` elements. Also, the databases in *Spider* carry primary and secondary key relations. The dataset contains a total of 10 181 questions across 138 different topics/themes, 5 693 complex SQL queries, and 200 multi-table databases in SQLite format, all annotated and compiled by 11 college students and the authors. However, only training and development sets are open-sourced for public use. The model must be submitted to the authors for evaluation to determine the performance of one's model on the test set; the result will be posted on the *Spider* leaderboard [14].

SParC: Due to the success and popularity of *Spider*, various versions, augmentations, and derivations have been introduced by researchers in the following years, all addressing a specific issue not covered in the original release. One of the most famous derived datasets released is *SParC* [15], which mainly addresses the issue of multi-turn interactions. The proponents of *SParC* argued that users would usually query databases in a sequential manner in real-world applications. Each query can then be understood contextually, depending on both the current and previous questions. *SParC* not only exhibits complex contextual dependencies, but it also requires generalization to new domains. This is due to its cross-domain nature, as well as the unseen databases during testing.

Over the years, *Spider* has become the de facto dataset for comparing different text-to-SQL models. For this reason, the authors have decided to use *Spider* as the benchmark dataset for the selected approaches.

B. Evaluation Metrics

This subsection discusses the common evaluation metrics used for the text-to-SQL task. All of the research to be discussed in later sections uses the *Spider* dataset. For this reason, we are going to give a brief discussion on the metrics mentioned in their paper.

***Component Matching**:* It is the average exact match of different SQL components between the predicted and gold query. Each component was decomposed into bags of several subcomponents and were then compared. Since each subcomponents is considered as sets to handle the "ordering issue."

```
G: SELECT avg(col1),max(col2),min(col1)
> (avg, min, col1) , (max, col2)

P: SELECT avg(col1), max(col2)
> (avg, col1) , (max, col2)
```

***Exact Matching Accuracy**:* The *exact matching accuracy* is determined by checking if the predicted query matches the gold query in its entirety. The predicted query can only be deemed correct if all of its components are accurate.

***Execution Accuracy**:* Relying solely on the *exact matching accuracy* of the model is not a great evaluation method. This is because there can be instances where erroneous negative assessments are produced during the generation of unique, accurate syntax structures. We also took into account the model's *execution accuracy* since each database in the Spider dataset contains its respective executable SQLite files. Nevertheless, in the process of determining the *execution accuracy*, there is the underlying possibility of erroneous positive assessments. This can occur when the generated SQL and the gold SQL are generally the same, albeit semantically different (e.g., `'NULL'`). Thus, it is better to compliment the *exact matching accuracy* of the model with its corresponding *execution accuracy*. Also, provided that the query contains `JOIN` and `GROUP`, numerous acceptable keys are also taken into account.

Each methodology's exact matching accuracy and execution accuracy were compared in this analysis. This is carried out to evaluate each model's performance in comparison to the others.

III. Review of Text-to SQL Approaches

This section provides an overview of all of the approaches and models presented in the selected publications. The authors reviewed works in the Spider leaderboard [14]. Six works that piqued the interest of the authors were then reviewed further and discussed in this study. The techniques and models are reviewed in chronological order, beginning with the earliest publication date and ending with the most recent. Their methods are summarized here for ease of discussion, but it is highly recommended to read all of the original works for in-depth discussion.

A. IRNet

Guo et al. [16] presented a neural method for complicated and cross-domain Text-to-SQL called *IRNet*. At that time their work was published, it was one of the first few that endeavored to work on the complicated *Spider* cross-domain dataset. The study was also one of the best known methods for introducing the use of intermediate representation, which will be covered in more detail in the sections to come.

Unlike previous approaches where models tend to produce SQL queries using end-to-end synthesis, IRNet broke down the task into three stages. First, schema linking was done between the question and schema to identify the column/table names mentioned in the question. This was then used to produce the SemQL query, which is an intermediate representation to bridge natural language questions and SQL. The SemQL query was then used to produce the SQL query with the help of domain knowledge.

The *SemQL* query, an intermediary representation created to bridge NL and SQL, is then synthesized by *IRNet* using a grammar-based neural network. *SemQL* is tree-structured, inspired from the works of Liang [17]. This means it is organized in such a way that it is simple to search for and discover information. Additionally, the tree-structure makes it simple to convert *SemQL* to SQL.

As input, the model accepts a query, a database schema, and the schema linking results. An NL encoder in the model converts the question into a series of hidden states. The model also includes a schema encoder that converts representations of columns and tables from the database schema. A decoder built inside the model produces *SemQL* queries. To create a *SemQL* query, the decoder works with three different kinds of actions: `APPLYRULE`, `SELECTCOLUMN`, and `SELECTTABLE`. Column selection in the decoder is done via a memory enhanced pointer network. Additionally, the decoder features a pointer network for choosing tables. The *SemQL* query decoding procedure is divided into two steps by the model's coarse-to-fine framework. A skeleton of the *SemQL* query is output in the first stage by a skeleton decoder. In the second stage, a detail decoder completes the skeleton's missing details.

B. RAT-SQL

The work of Wang et al. [18] presents *RAT-SQL*, a unified framework for a text-to-SQL encoder that addresses schema encoding, schema linking, and feature representation. It analyzes the less explored concept of schema linking – the challenge of aligning table and column schema to entities in the question. And tries to resolve issues from prior works of Bogin et al. [19] in schema encoding such as 1) lack of contextualization between the schema and the question and

2) the restricted propagation of information to the predefined graph of foreign key relations during schema encoding.

The framework is based on the relation-aware self-attention mechanism. Relation-aware self-attention is a way to encode semi-structured input sequences, which takes into account both the pre-existing relational structure of the input, and also the relationships between elements in the same sequence. This framework provides a natural way to implement schema embedding and linking.

It starts off with an initial representation for every node with either a pre-trained `GloVe` embedding followed by a bi-directional LSTM or a pre-trained language model(PLM) such as `BERT`. This is then fed into the relation-aware transformer (RAT) layers. The decoder used followed the tree-structured architecture of Yin et ., [20] which used an LSTM layer to output a sequence of decoder action. The actions are `APPLYRULE` – that expands the previous node into a grammar rule – or, `SELECTCOLUMN` and `SELECTABLE` – which chooses a column/table when completing a leaf node

C. BRIDGE

Lin et al. [21] further improved on the idea of schema encoding by leveraging extracted database cell values called *anchor text* which were used to augment the hybrid sequence. By leveraging the value-set of each field in the database, the model can capture the dependencies between the question and database schema – which was not always possible when only using table/field names. Consider a table with field name `Property_type` and value sets `[House, Apartments, Commercial]`, the model can find the relationship between the question and the schema *"List of apartments"* when the value sets are encoded along with the question instead of just using the field name.

$$X = [CLS], Q, [SEP], [T], t_1, [C], c_{11}, [Anchor], [V]...,$$
$$c_{1|T_1|}, [T], t_2, [C], c_{21}..[C]..c_{N|T_N|}[SEP]$$

Anchor text were obtained by performing a fuzzy string matching between the question and picklist of each field in the database. It was then inserted after the corresponding field name in the question-schema representation *X* followed by a special token `[V]`. The representation was then encoded with a BERT-based language model followed by a bi-drectional LSTM layer to obtain the base encoding. The question segment was passed through another bi-LSTM layer to obtain the question encoding. Dense look-up features were also trained which include information on whether a field is a primary key or foreign key and the data type of each field.

The encoded features were then passed through an LSTM-based pointer-generator with multi-head attention. At each step, the decoder either generates a token from the vocabulary *V* , copies a token from the question *Q*, or copies a schema component from *S*. Unlike other models that only predicts SQL syntax skeleton, the decoder can directly generate a token to produce the SQL query.

D. SmBop

Recent years have seen widespread adoption of the encoder-decoder design for semantic parsing. While the encoder side has seen improvements with the help of transformer-based architectures and pre-training techniques, research on the decoder side has remained stagnant over the years. Rubin et al., [22] proposed a bottom-up semantic decoder far different from the accustomed top-down manner [20]. Their decoder showed faster runtime complexity – logarithmic rather than linear – and outperformed pre-existing decoder architectures.

Since their work is mostly focused on developing a decoder, they just used a pre-existing encoder namely, the RAT-SQL coupled with a specialized pre-trained language model `GRAPPA`. GRAPPA is based on `RoBERTa` and uses relational algebra as an intermediate representation of the SQL queries. The encoder is used to initialize a beam, which is a tree that includes either schema constants or DB values. The beam is scored independently and in parallel. Next is the search procedure the model uses to find the top-scoring tree. At each step, the model looks at the trees on the frontier and scores them based on how well they match the representation of the input question. The model then chooses the top k trees and computes a new representation. Finally, the model returns the top-scoring tree from the final decoding step. The *SmBop* algorithm is similar to beam search and chart parsing. The algorithm holds the top k trees of a fixed height in beam search. In chart parsing, trees at step $t+1$ are computed from trees that were found at step t.

E. ShadowGNN

A newer approach was to use Graph Neural Networks [23] in order to abstract the natural language question and schema. Originally introduced in the text-to-SQL back in 2019 [24], Chen et al. [25] improved on previous works with the *ShadowGNN* model which used graph projection neural network (GPNN) on their encoder to produce then state-of-the-art results in the Spider dataset. When trained with extremely limited data, it outperforms other model in exact match accuracy.

The Graph Projection Neural Network (GPNN) takes in three inputs namely, the abstract schema, the semantic schema, and the natural language question. The abstract schema are one-hot vectors that represents the type(table or column) while the natural language question and semantic schema are concatenated and fed through the pre-trained language model `RoBERTa`. For each GPNN layer the attention between the question and semantic schema was computed. Schema linking was then added to the attention probabilities. It is a one-hot representation of seven match types, namely Table Exact Match, Table Partial Match, Column Exact Match, Column Partial Match, Column Value Exact Match, Column Value Partial Match, and No Match. The maximum value of the attention probability was then computed. This was then multiplied to the abstract schema mentioned earlier.

The question and schema representation were then passed through a relation aware transformer (RAT) before being

decoded similar to IRNet. A SemQL query was predicted with the help of a skeleton decoder followed by a detail decoder to fill in missing details using table and column names.

F. LGESQL

Cao et al. [26] proposed the use of Line Graphs which were able to capture rich semantics embbeded in topological structure of edges. They were also able to distinguish local and nonlocal relations for each node that previous graph encoding methods were not capable of doing.

The question and schema are first concatenated into a sequence and is fed through a pre-trained language model. These are then fed through three layers of BiLSTMs in order to get the graph inputs. The graph inputs were then fed into a stack of L dual relational graph attention networks (RGAT) layers. Each RGAT tries to capture the structure of the original graph, and line graph respectively. The steps on how each RGAT layers compute the edge and node features are discussed in detail in their paper. They then used a grammar-based syntactic decoder in order to output the abstract syntax tree.

IV. Analysis

This section analyzes the performance, and design similarities and differentiation of the different model architectures mentioned earlier.

A. Performance

The execution accuracy and exact match accuracy of the different models are summarized in Table I. The test scores were obtained from the official Spider leaderboard [14].

TABLE I
TEST RESULTS

Model	Dev Set		Test Set	
	Exec	Exact	Exec	Exact
IRNet+BERT	-	61.9	-	54.7
RATSQL+BERT	-	69.7	-	65.6
BRIDGE(ensemble)	-	71.1	68.3	67.5
SmBop	75.3*	74.7	71.1	69.5
ShadowGNN	-	72.3	-	66.1
LGESQL(Electra)	34.2*	75.1	-	72.0

Scores with * were obtained from the official Github repository of the model made public by the authors

From the introduction of IRNet in 2019, the exact match accuracy has increased throughout different advances and adoption of similar ideas. Most of the time, these models only needs to gain around $\pm 2\%$ in exact match accuracy to obtain state-of-the-art.

Surprisingly, most models only provide their exact match accuracy and only a few publish their execution accuracy. This is also the same situation among the different models included in the official Spider leaderboard. LGESQL [26] – which has the highest exact match accuracy among the six models discussed – has a significantly lower execution accuracy when compared to other models such as SmBop and BRIDGE.

B. Encoder-Decoder Model

One observation of the different models discussed earlier is that they all follow the same encoder-decoder architecture. While some models develop their encoder and decoder from scratch, others adopted already existing components while they work on the other. SmBop and DuoRAT [27] (not discussed in this paper) for example adopted the encoder of RAT-SQL while they develop their decoder. RAT-SQL and LGESQL followed a similar approach to Yin et al. [20] for their model's decoder.

Encoder: The encoder usually follow a three step process. The natural language question is first concatenated together with the database schema. This is then passed through a pre-trained language model such as `BERT` or `GraPPa`. The third step is to use a neural architecture that can be built using deep neural layers such as Bi-LSTM, relation aware transformer, or graph neural network to obtain the rich vector representation.

Decoder: Some decoder can generate SQL directly similar to the Seq2Seq model such as *BRIDGE* [21]. However, this may produce inexecutable SQL queries. To combat this, some decoders uses a grammar-based approach which generate abstract syntax trees or intermediate representation such as *SemQL* first before being decoded back to SQL.

The difference in the number of advancements per component was also apparent. As mentioned by Rubin et al., [22] a large number of text-to-SQL research only focused on the encoder while the decoder only received a few improvements. Future research should also consider further developing or introducing new decoder architectures since this may lead to increased performance and a reduction in training and inference time as shown by SmBop.

C. Use of Pre-Trained Language Models

The us of pre-trained language models were also necessary to further improve the performance of each models. Embeddings from these large PLMs significantly improves the output when compared to static embeddings such as `GloVe` as shown in Table II.

TABLE II
EFFECTS OF PRE-TRAINED LANGUAGE MODELS ON EXACT MATCH ACCURACY

Model	Language Model	Dev	Test
IRNet	GloVe	53.2	46.7
	BERT	61.9	54.7
RAT-SQL	GloVe	62.7	57.2
	BERT	69.7	65.6
LGESQL	GloVe	67.6	62.8
	BERT	74.1	68.3
	ELECTRA	75.1	72.0

It can be seen how the use of BERT-based language model can improve performance by $7-8\%$ on exact match accuracy. Because of this, specialized language models for the text-to-SQL task were also developed such as `GraPPa` [28] and `STRUG` [29] to further improve the performances as shown in Table III.

TABLE III
PERFORMANCE OF RAT-SQL USING SPECIALIZED PRE-TRAINED LANGAUGE MODELS

Model	Exact	Exec	Exact(Test)
BERT	69.8 ± 0.8	72.3 ± 0.6	-
GraPPa	73.4	-	69.6
STRUG (Human Assisted)	72.7 ± 0.7	75.5 ± 0.8	68.4

Data taken from the work of Deng et al. [29]

One downside of these specialized pre-trained language models however is that they are only available at the moment for the English language. `BERT-base` models are available for other languages but does not offer the same performance to specialized models such as `GraPPa` or `STRUG`. Questions in other languages should be able to properly encoded along with English schema (current industry practice) [30].

D. Intermediate Representations

Some of the models discussed used an intermediate representation to further improve the alignment because of the mismatch between natural language and SQL queries. IRNet, and ShadowGNN used SemQL [16] while SmBop adopted pre-existing *relational algebra*.

SemQL is a powerful tool for increasing the performance of text-to-SQL models. However, it has not fully eliminated the mismatch between natural language questions and SQL queries. For example, an end-user does not explicitly state in their question whether two conditions are disjoint whereas in SQL the `INTERSECT` clause does. Not only this, but it can not support the self join using the `FROM` clause.

Gan et al. introduced *NatSQL* in 2021 to simplify and provide better alignment between natural language questions and SQL queries. NatSQL managed to (1) dispense with operators and keywords like `GROUP BY, FROM, JOIN ON` etc.; (2) removed the need for set operators and nested sub-queries ; (3) made schema linking easier by reducing the amount of required schema items. The use of NatSQL improved the performance of select text-to-SQL models when compared to previous intermediate representations as shown in Table IV

TABLE IV
PERFORMANCE OF NATSQL

Approach	Exact	Execution
IRNet + SemQL	51.8	-
IRNet + NatSQL	52.9	52.6
RAT-SQL(BERT) + IR(RAT-SQL)	71.8	-
RAT-SQL(BERT) + NatSQL	73.7	74.6

Data taken from the work of Gan et al. [31]

While there were no limitations explicitly stated on the use of relational algebra for NLIDB, there is no definite study that compares the usage of relational algebra over SemQL and NatSQL. Direct comparison is also difficult since the choice of intermediate language highly affects the decoder architecture of the system.

V. CONCLUSIONS

This study reviewed multiple different approaches to text-to-SQL semantic parsing. The field has immensely progressed in the recent years, as highlighted by the increase in performance of the models reviewed. Although each approach follows an encoder-decoder architecture, it was evident that the majority of the recent advancements focused on the encoder side. This leaves a huge opportunity for decoder improvement in future works. It was also notable that using pre-trained language models improved the performances of the models, and should be worth considering for future works. On the other hand, the choice of intermediate representations is completely subjective.

As future work, the authors can experiment with large language models more advanced than OpenAI's GPT-3 for text-to-SQL semantic parsing. Another area to consider is the change in SQL dialect, such as MySQL and PostgeSQL, since the Spider dataset uses SQLite dialect and syntax.

ACKNOWLEDGEMENT

The authors would like to thank and acknowledge the Computer Software Division of DOST-ASTI, mainly the iTANONG project staff, for their unwavering support and contributions to this study.

REFERENCES

[1] Alvaro Monjaras, Enrique Bcndezu, and Carlos Raymundo. Decision tree model to support the successful selection of a database engine for novice database administrators. pages 353–357. Institute of Electrical and Electronics Engineers Inc., 5 2019.

[2] Zenan Ning, Dongjiao Zhang, Lei Zhang, Hongzhi Yu, and Fucheng Wan. Review of question answering technology based on text to sql. pages 143–146. Institute of Electrical and Electronics Engineers Inc., 1 2021.

[3] Diptikalyan Saha, Avrilia Floratou, Karthik Sankaranarayanan, Umar Farooq Minhas, Ashish R. Mittal, and Fatma Özcan. Athena. *Proceedings of the VLDB Endowment*, 9:1209–1220, 8 2016.

[4] Pooja Malhotra, Yash Kapadia, Krishna Saboo, and Ankita Sarda. Question-answering system using natural language processing with nlidb approach — international journal of current research. *International Journal of Current Research*, 9:57575–57577, 9 2017.

[5] Syed Ahmad Chan Bukhari, Hafsa Shareef Dar, M. Ikramullah Lali, Fazel Keshtkar, Khalid Mahmood Malik, and Seifedine Kadry. Frameworks for querying databases using natural language: A literature review – nlp-to-db querying frameworks. *International Journal of Data Warehousing and Mining (IJDWM)*, 17:21–38, 2021.

[6] Orest Gkini, Theofilos Belmpas, Georgia Koutrika, and Yannis Ioannidis. An in-depth benchmarking of text-to-sql systems. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, page 632–644, New York, NY, USA, 2021. Association for Computing Machinery.

[7] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. 8 2017.

[8] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task, 2018.

[9] Shanza Abbas, Muhammad Umair Khan, Scott Uk Jin Lee, Asad Abbas, and Ali Kashif Bashir. A review of nlidb with deep learning: Findings, challenges and open issues. *IEEE Access*, 10:14927–14945, 2022.

[10] Radu Cristian Alexandru Iacob, Florin Brad, Elena-Simona Apostol, Ciprian-Octavian Truică, Ionel Alexandru Hosu, and Traian Rebedea. Neural approaches for natural language interfaces to databases: A survey. pages 381–395. Association for Computational Linguistics (ACL), 1 2020.

[11] Muhammad Shahzaib Baig, Azhar Imran, Aman Ullah Yasin, Abdul Haleem Butt, and Muhammad Imran Khan. Natural language to sql queries: A review. *International Journal of Innovations in Science Technology*, 4:147–162, 2022.

[12] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, Fei Huang, and Yongbin Li. A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions. *arXiv e-prints*, page arXiv:2208.13629, August 2022.

[13] Amandalynne Paullada, Inioluwa Deborah Raji, Emily M. Bender, Emily Denton, and Alex Hanna. Data and its (dis)contents: A survey of dataset development and use in machine learning research. *Patterns*, 2(11):100336, 2021.

[14] Spider: Yale semantic parsing and text-to-sql challenge.

[15] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. Sparc: Cross-domain semantic parsing in context, 2019.

[16] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. Towards complex text-to-sql in cross-domain database with intermediate representation. *CoRR*, abs/1905.08205, 2019.

[17] Percy Liang. Lambda dependency-based compositional semantics. *CoRR*, abs/1309.4408, 2013.

[18] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: relation-aware schema encoding and linking for text-to-sql parsers. *CoRR*, abs/1911.04942, 2019.

[19] Ben Bogin, Jonathan Berant, and Matt Gardner. Representing schema structure with graph neural networks for text-to-SQL parsing. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4560–4565, Florence, Italy, July 2019. Association for Computational Linguistics.

[20] Pengcheng Yin and Graham Neubig. A syntactic neural model for general-purpose code generation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 440–450, Vancouver, Canada, July 2017. Association for Computational Linguistics.

[21] Xi Victoria Lin, Richard Socher, and Caiming Xiong. Bridging textual and tabular data for cross-domain text-to-sql semantic parsing. In *FINDINGS*, 2020.

[22] Ohad Rubin and Jonathan Berant. Smbop: Semi-autoregressive bottom-up semantic parsing. *CoRR*, abs/2010.12412, 2020.

[23] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.

[24] Ben Bogin, Jonathan Berant, and Matt Gardner. Representing schema structure with graph neural networks for text-to-SQL parsing. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4560–4565, Florence, Italy, July 2019. Association for Computational Linguistics.

[25] Zhi Chen, Lu Chen, Yanbin Zhao, Ruisheng Cao, Zihan Xu, Su Zhu, and Kai Yu. ShadowGNN: Graph projection neural network for text-to-SQL parser. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5567–5577, Online, June 2021. Association for Computational Linguistics.

[26] Ruisheng Cao, Lu Chen, Zhi Chen, Yanbin Zhao, Su Zhu, and Kai Yu. LGESQL: Line graph enhanced text-to-SQL model with mixed local and non-local relations. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2541–2555, Online, August 2021. Association for Computational Linguistics.

[27] Torsten Scholak, Raymond Li, Dzmitry Bahdanau, Harm de Vries, and Chris Pal. DuoRAT: Towards simpler text-to-SQL models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1313–1321, Online, June 2021. Association for Computational Linguistics.

[28] Tao Yu, Chien-Sheng Wu, Xi Victoria Lin, Bailin Wang, Yi Chern Tan, Xinyi Yang, Dragomir R. Radev, Richard Socher, and Caiming Xiong. Grappa: Grammar-augmented pre-training for table semantic parsing. *CoRR*, abs/2009.13845, 2020.

[29] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. Structure-grounded pretraining for text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021.

[30] Qingkai Min, Yuefeng Shi, and Yue Zhang. A pilot study for chinese SQL semantic parsing. *CoRR*, abs/1909.13293, 2019.

[31] Yujian Gan, Xinyun Chen, Jinxia Xie, Matthew Purver, John R. Woodward, John H. Drake, and Qiaofu Zhang. Natural SQL: making SQL easier to infer from natural language specifications. *CoRR*, abs/2109.05153, 2021.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.